

Introduction to Artificial Intelligence

Logical Agents

Matteo Acclavio

19/03/2026

Plan

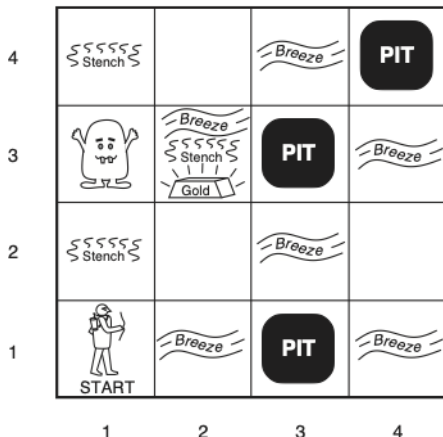
So far:

- We have seen how to describe an agent and its environment
- We have seen how to plan by considering the state as 'atomic'
- We have seen that we can factor the state

Today:

- Define logical agents
- Introduce the language of logic
- Introduce the notion of inference

The Wumpus World



- Performance measure: +1000 for gold, -1 for each action, -1000 for dying
- Environment: Known, deterministic, sequential, static, partially observable
- Actuators (allowing to): move forward, turn left, turn right, grab, exit
- Sensors (allowing to sense): stench, breeze, glitter

Known: 4x4 grid, the square adjacent to the Wumpus stinks, the square adjacent to a pit is breezy, the square with gold glitters.

Partially observable: the agent can only sense the current square; at the beginning, the agent does not know the environment.

Logical agent

Main idea:

- The agent maintains a **knowledge base (KB)**

Logical agent

Main idea:

- The agent maintains a **knowledge base** (KB)
- The agent can **infer** new knowledge from the KB

Logical agent

Main idea:

- The agent maintains a **knowledge base** (KB)
- The agent can **infer** new knowledge from the KB
- The agent can **act** based on the KB

Logical agent

Main idea:

- The agent maintains a **knowledge base** (KB)
- The agent can **infer** new knowledge from the KB
- The agent can **act** based on the KB

Yes, but how?

Knowledge base and inference

Main idea:

- Knowledge base: a set of sentences in a **formal language** containing:
 - Background knowledge about the world
 - Knowledge about the current state of the world
 - Knowledge about the effects of actions

Knowledge base and inference

Main idea:

- Knowledge base: a set of sentences in a **formal language** containing:
 - Background knowledge about the world
 - Knowledge about the current state of the world
 - Knowledge about the effects of actions
- A mechanism to update KB when the agent receives new percepts
 - Add new sentences to the KB when the agent receives new percepts
 - Add new sentences to the KB when the agent performs an action

Knowledge base and inference

Main idea:

- Knowledge base: a set of sentences in a **formal language** containing:
 - Background knowledge about the world
 - Knowledge about the current state of the world
 - Knowledge about the effects of actions
- A mechanism to update KB when the agent receives new percepts
 - Add new sentences to the KB when the agent receives new percepts
 - Add new sentences to the KB when the agent performs an action
- A mechanism that derives new sentences from the KB

Knowledge Representation Language

- Syntax: defines the symbols and the rules for constructing sentences which 'make sense'

Knowledge Representation Language

- Syntax: defines the symbols and the rules for constructing sentences which 'make sense'
- Semantics: defines the meaning of the sentences

Knowledge Representation Language

- Syntax: defines the symbols and the rules for constructing sentences which 'make sense'
- Semantics: defines the meaning of the sentences

Logic!

Syntax

‘Make sense’ = ‘well-formed’ = ‘grammatically correct’

'Make sense' = 'well-formed' = 'grammatically correct'

- Syntax of arithmetic expressions: $x + y = 7$ makes sense, but $7x+ = y$ does not make sense

'Make sense' = 'well-formed' = 'grammatically correct'

- Syntax of arithmetic expressions: $x + y = 7$ makes sense, but $7x + = y$ does not make sense
- Syntax of English sentences: 'The cat is on the table' makes sense, but 'on cat the The is table' does not make sense

‘Make sense’ = ‘well-formed’ = ‘grammatically correct’

- Syntax of arithmetic expressions: $x + y = 7$ makes sense, but $7x + = y$ does not make sense
- Syntax of English sentences: ‘The cat is on the table’ makes sense, but ‘on cat the The is table’ does not make sense
- Syntax of palindromic words: ‘madam’ makes sense, but ‘hello’ does not make sense

Syntax

'Make sense' = 'well-formed' = 'grammatically correct'

- Syntax of arithmetic expressions: $x + y = 7$ makes sense, but $7x + = y$ does not make sense
- Syntax of English sentences: 'The cat is on the table' makes sense, but 'on cat the The is table' does not make sense
- Syntax of palindromic words: 'madam' makes sense, but 'hello' does not make sense

Idea: give a grammar to define the syntax of a language

Attribute meaning to sentences

Attribute meaning to sentences

- Semantics of arithmetic expressions: $x + y = 7$ has a meaning 'if you add x and y you get 7'

Attribute meaning to sentences

- Semantics of arithmetic expressions: $x + y = 7$ has a meaning 'if you add x and y you get 7'
- Semantics of English sentences: 'The cat is on the table' has a meaning 'there is a cat and there is a table and the cat is on the table'

Attribute meaning to sentences

- Semantics of arithmetic expressions: $x + y = 7$ has a meaning 'if you add x and y you get 7'
- Semantics of English sentences: 'The cat is on the table' has a meaning 'there is a cat and there is a table and the cat is on the table'

Idea: give a 'possible world' where we can interpret any (syntactically correct) sentence of the language, and tell if the sentence is true or false in that world

Syntax and semantics for the Wumpus World

Syntax and semantics for the Wumpus World

Syntax:

- 'there is a pit in square (x, y) '

Syntax and semantics for the Wumpus World

Syntax:

- 'there is a pit in square (x, y) '
- 'there is a Wumpus in square (x, y) '

Syntax and semantics for the Wumpus World

Syntax:

- 'there is a pit in square (x, y) '
- 'there is a Wumpus in square (x, y) '
- 'there is a breeze in square (x, y) '

Syntax and semantics for the Wumpus World

Syntax:

- 'there is a pit in square (x, y) '
- 'there is a Wumpus in square (x, y) '
- 'there is a breeze in square (x, y) '
- 'there is a stench in square (x, y) '

Syntax and semantics for the Wumpus World

Syntax:

- 'there is a pit in square (x, y) '
- 'there is a Wumpus in square (x, y) '
- 'there is a breeze in square (x, y) '
- 'there is a stench in square (x, y) '
- 'there is a glitter in square (x, y) '

Syntax and semantics for the Wumpus World

Syntax:

- 'there is a pit in square (x, y) '
- 'there is a Wumpus in square (x, y) '
- 'there is a breeze in square (x, y) '
- 'there is a stench in square (x, y) '
- 'there is a glitter in square (x, y) '

Note: 'there is a pit in square $(5, 6)$ ' may make sense or not, depending on the restriction we decide on the language

Syntax and semantics for the Wumpus World

Syntax:

- 'there is a pit in square (x, y) '
- 'there is a Wumpus in square (x, y) '
- 'there is a breeze in square (x, y) '
- 'there is a stench in square (x, y) '
- 'there is a glitter in square (x, y) '

Note: 'there is a pit in square $(5, 6)$ ' may make sense or not, depending on the restriction we decide on the language

Semantics:

- A 'possible world' is an assignment of truth values to the propositions in the language

Being Formal

Models and Satisfiability

Definition:

A **model** for a syntax is an assignment of truth values to all the propositions in the language.

$$\mathfrak{M} : \text{Propositions} \rightarrow \{\top, \perp\}$$

- A sentence α is **satisfiable** in a model $\mathfrak{M}$ if it is true in that model [Notation: $\mathfrak{M} \models \alpha$]
- A sentence α is **valid** if it is true in all models [Notation: $\models \alpha$]
- A sentence α is **unsatisfiable** if it is false in all models

Entailment

Definition:

A sentence α **entails** a sentence β if every model that satisfies α also satisfies β

$$\alpha \models \beta \text{ iff } (M \models \alpha \text{ implies } M \models \beta \text{ for every model } M)$$

Two sentences α and β are **logically equivalent** if $\alpha \models \beta$ and $\beta \models \alpha$
 $\alpha \equiv \beta$ iff ($\alpha \models \beta$ and $\beta \models \alpha$)

Entailment

Definition:

A sentence α **entails** a sentence β if every model that satisfies α also satisfies β

$$\alpha \models \beta \text{ iff } (M \models \alpha \text{ implies } M \models \beta \text{ for every model } M)$$

Two sentences α and β are **logically equivalent** if $\alpha \models \beta$ and $\beta \models \alpha$
 $\alpha \equiv \beta$ iff ($\alpha \models \beta$ and $\beta \models \alpha$)

How to check if $\alpha \models \beta$?

Entailment

Definition:

A sentence α **entails** a sentence β if every model that satisfies α also satisfies β

$$\alpha \models \beta \text{ iff } (M \models \alpha \text{ implies } M \models \beta \text{ for every model } M)$$

Two sentences α and β are **logically equivalent** if $\alpha \models \beta$ and $\beta \models \alpha$
 $\alpha \equiv \beta$ iff ($\alpha \models \beta$ and $\beta \models \alpha$)

How to check if $\alpha \models \beta$?

- Brute force (model checking):
 - enumerate all models
 - check if α is true in the model
 - if α is true, check if β is true

Inference

An ***inference algorithm*** is a procedure that takes as input a sentence α and a sentence β and returns 'yes' if $\alpha \models \beta$ and 'no' otherwise.

$$\alpha \vdash_{\text{MyAlgo}} \beta$$

Inference

An ***inference algorithm*** is a procedure that takes as input a sentence α and a sentence β and returns 'yes' if $\alpha \models \beta$ and 'no' otherwise.

$$\alpha \vdash_{\text{MyAlgo}} \beta$$

An inference algorithm is

Inference

An *inference algorithm* is a procedure that takes as input a sentence α and a sentence β and returns 'yes' if $\alpha \models \beta$ and 'no' otherwise.

$$\alpha \vdash_{\text{MyAlgo}} \beta$$

An inference algorithm is

- *sound* (or *truth-preserving*) if it only returns 'yes' when $\alpha \models \beta$.

Inference

An ***inference algorithm*** is a procedure that takes as input a sentence α and a sentence β and returns 'yes' if $\alpha \models \beta$ and 'no' otherwise.

$$\alpha \vdash_{\text{MyAlgo}} \beta$$

An inference algorithm is

- ***sound*** (or ***truth-preserving***) if it only returns 'yes' when $\alpha \models \beta$.
- ***complete*** if it returns 'yes' every $\alpha \models \beta$.

Inference

An ***inference algorithm*** is a procedure that takes as input a sentence α and a sentence β and returns 'yes' if $\alpha \models \beta$ and 'no' otherwise.

$$\alpha \vdash_{\text{MyAlgo}} \beta$$

An inference algorithm is

- ***sound*** (or ***truth-preserving***) if it only returns 'yes' when $\alpha \models \beta$.
- ***complete*** if it returns 'yes' every $\alpha \models \beta$.

Brute force is an inference algorithm, but it is not efficient.

Logic

Propositional Logic

Syntax of propositional logic:

We fix a set of propositional symbols $\mathbb{A}$, and we defines formulas as follows:

$$A, B ::= p \mid \neg A \mid A \wedge B \mid A \vee B$$

Propositional Logic

Syntax of propositional logic:

We fix a set of propositional symbols $\mathbb{A}$, and we defines formulas as follows:

$$A, B ::= p \mid \neg A \mid A \wedge B \mid A \vee B \mid A \rightarrow B$$

Propositional Logic

Syntax of propositional logic:

We fix a set of propositional symbols $\mathbb{A}$, and we defines formulas as follows:

$$A, B ::= p \mid \neg A \mid A \wedge B \mid A \vee B \mid A \rightarrow B \mid A \leftrightarrow B$$

Propositional Logic

Syntax of propositional logic:

We fix a set of propositional symbols $\mathbb{A}$, and we defines formulas as follows:

$$A, B ::= p \mid \neg A \mid A \wedge B \mid A \vee B \mid A \rightarrow B \mid A \leftrightarrow B$$

Semantics of propositional logic:

We define the semantics of propositional logic by:

- giving a truth value for each propositional symbol in $\mathbb{A}$ in a model $\mathfrak{M}$

Deduction Theorem, and how to check entailment

An important result in propositional logic is the ***deduction theorem***: the connective for implication captures the notion of entailment.

Theorem: Deduction Theorem

For any sentences α and β ,

$$\alpha \models \beta \quad \text{iff} \quad \models \alpha \rightarrow \beta$$

Deduction Theorem, and how to check entailment

An important result in propositional logic is the ***deduction theorem***: the connective for implication captures the notion of entailment.

Theorem: Deduction Theorem

For any sentences α and β ,

$$\alpha \models \beta \quad \text{iff} \quad \models \alpha \rightarrow \beta$$

Theorem:

For any sentences α and β ,

$$\alpha \models \beta \quad \text{iff} \quad \not\models \alpha \wedge \neg \beta$$

Inference algorithms

Automatizing inference

The most famous inference rule is the *modus ponens*:

$$\text{MP} \frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Automatizing inference

The most famous inference rule is the *modus ponens*:

$$\text{MP} \frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Why use inference rules instead of model checking?

- No need of checking all models
- Rules are often more intuitive than truth tables
- The search space is narrowed down by the rules

Example of use the Modus Ponens

$$\text{MP} \frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

When we decided that $[2, 2]$ was safe in the Wumpus world

Forward and backward chaining

Start with a knowledge base KB and a query α .

- **Forward chaining:** start with KB and apply inference rules to add new facts, until we derive α or we cannot derive anything else.

Forward and backward chaining

Start with a knowledge base KB and a query α .

- **Forward chaining:** start with KB and apply inference rules to add new facts, until we derive α or we cannot derive anything else.
 - Soundness
 - Completeness

Forward and backward chaining

Start with a knowledge base KB and a query α .

- **Forward chaining:** start with KB and apply inference rules to add new facts, until we derive α or we cannot derive anything else.
 - Soundness
 - Completeness
- **Backward chaining:** start with α and apply inference rules backwards to find new subgoals, until we find a contradiction or we cannot find any new subgoal.

Forward and backward chaining

Start with a knowledge base KB and a query α .

- **Forward chaining:** start with KB and apply inference rules to add new facts, until we derive α or we cannot derive anything else.
 - Soundness
 - Completeness
- **Backward chaining:** start with α and apply inference rules backwards to find new subgoals, until we find a contradiction or we cannot find any new subgoal.
 - Soundness
 - Completeness

Summary

Book references:

- Chapter until 7.5.1 (included)

Next TA session:

- More exercises on search [Chapter 3 and 4 so far]

Next lectures:

- Automated reasoning and First-order logic (week 15, 2 classes)
- Tutorial on MiniZinc and Prolog (week 16)

Happy Easter holidays!